

SCR2043 OPERATING SYSTEMS

Name : TAN ZHI MING
Student ID : A23CS0189
Section : 03

Marks

This lab assessment is designed to test your understanding and skills on some basic concepts and tools related to process monitoring and management in operating system. Please follow the instructions carefully and submit your answers in this word document and rename the file as **os-lab-assessment02-studentname-matricno.docx**.

Essential Steps Before Starting Lab Assessment 2:

1. Download necessary source codes:

Use the `wget` command to retrieve the following source code files to your Linux (or WSL or MacOS) environment:

```
wget -O mainprocess.c https://rebrand.ly/mainprocess_c
wget -O subprocess1.c https://rebrand.ly/subprocess1_c
wget -O subprocess2.c https://rebrand.ly/subprocess2_c
```

2. Compile the source files:

Use the `gcc` compiler to create executable files from the source code.

```
gcc mainprocess.c -o mainprocess
gcc subprocess1.c -o subprocess1
gcc subprocess2.c -o subprocess2
```

3. Execute the dummy processes:

Run all the dummy processes

```
./mainprocess &
```

Press **enter** two times.

4. The dummy processes are running for 2 hours. If you took longer than 2 hours on questions 1-9, please restart the main process with `./mainprocess &`.

Lab Assessment 2 : Linux Process Monitoring and Management

Instructions:

1. Carefully execute each command as instructed in the questions.
2. Write down the exact command used for each task.
3. Capture a screenshot of the command's output.

Question 1

Use the `ps` command with the appropriate option to display a complete list of all running processes within the Linux operating system.

Command				
<code>ps -e</code>				
Output				
	996	tty1	00:00:00	mainprocess
	997	tty1	00:00:00	mainprocess
	998	tty1	00:00:00	mainprocess
	999	tty1	00:00:00	subprocess1
	1000	tty1	00:00:00	subprocess1
	1001	tty1	00:00:00	subprocess2
	1002	tty1	00:00:00	subprocess2
	1003	tty1	00:00:00	subprocess2

Question 2

Employ the `ps` command with necessary options to unveil comprehensive details about each running process.

Command										
<code>ps -ef grep -E 'mainprocess subprocess'</code>										
Output										
zhiming@sechr2043:~\$ ps -ef grep -E 'mainprocess subprocess'										
zhiming	996	933	0	15:05	tty1	00:00:00	./mainprocess			
zhiming	997	996	0	15:05	tty1	00:00:00	./mainprocess			
zhiming	998	996	0	15:05	tty1	00:00:00	./mainprocess			
zhiming	999	997	0	15:05	tty1	00:00:00	./subprocess1			
zhiming	1000	997	0	15:05	tty1	00:00:00	./subprocess1			
zhiming	1001	998	0	15:05	tty1	00:00:00	./subprocess2			
zhiming	1002	998	0	15:05	tty1	00:00:00	./subprocess2			
zhiming	1003	998	0	15:05	tty1	00:00:00	./subprocess2			
zhiming	1006	933	0	15:06	tty1	00:00:00	grep --color=auto -E mainprocess subprocess			

Question 3

Use the `ps` command with some tools to only list processes named "subprocess" and show some info about them.

Command	
<code>ps -ef grep -E 'subprocess'</code>	
Output	
<pre>zhiming@secr2043:~\$ ps -ef grep -E 'subprocess' zhiming 999 997 0 15:05 tty1 00:00:00 ./subprocess1 zhiming 1000 997 0 15:05 tty1 00:00:00 ./subprocess1 zhiming 1001 998 0 15:05 tty1 00:00:00 ./subprocess2 zhiming 1002 998 0 15:05 tty1 00:00:00 ./subprocess2 zhiming 1003 998 0 15:05 tty1 00:00:00 ./subprocess2 zhiming 1008 933 0 15:07 tty1 00:00:00 grep --color=auto -E subprocess</pre>	

Question 4

Execute the `ps` command, specifying options that reveal only the following columns:

- Process ID (pid)
- Owner of the process (user)
- CPU percentage (pcpu)
- Memory percentage (pmem)
- Command (cmd)

Command	
<code>ps -eo pid,user,pcpu,pmem,cmd grep -E 'subprocess'</code>	
Output	
<pre>zhiming@secr2043:~\$ ps -eo pid,user,pcpu,pmem,cmd grep -E 'subprocess' 999 zhiming 0.0 0.1 ./subprocess1 1000 zhiming 0.0 0.1 ./subprocess1 1001 zhiming 0.0 0.1 ./subprocess2 1002 zhiming 0.0 0.1 ./subprocess2 1003 zhiming 0.0 0.1 ./subprocess2 1019 zhiming 0.0 0.2 grep --color=auto -E subprocess</pre>	

Question 5

Building on the `ps` command used in Question 4, can you add an option to sort the listed processes by their memory usage (`pmem`)?

Command
<code>ps -eo pid,user,pcpu,pmem,cmd --sort=pmem grep -E 'subprocess'</code>
Output
<pre>zhiming@secr2043:~\$ ps -eo pid,user,pcpu,pmem,cmd --sort=pmem grep -E 'subprocess' 999 zhiming 0.0 0.1 ./subprocess1 1000 zhiming 0.0 0.1 ./subprocess1 1001 zhiming 0.0 0.1 ./subprocess2 1002 zhiming 0.0 0.1 ./subprocess2 1003 zhiming 0.0 0.1 ./subprocess2 1027 zhiming 0.0 0.2 grep --color=auto -E subprocess</pre>

Question 6

Construct a command using `ps`, suitable options, and any additional tools to visualize the hierarchical structure (tree-like) of the following processes:

- "mainprocess"
- "subprocess1"
- "subprocess2"

Command
<code>ps --forest -C mainprocess -C subprocess1 -C subprocess2</code>
Output
<pre>zhiming@secr2043:~\$ ps --forest -C mainprocess -C subprocess1 -C subprocess2 PID TTY TIME CMD 996 tty1 00:00:00 mainprocess 997 tty1 00:00:00 _ mainprocess 999 tty1 00:00:00 _ subprocess1 1000 tty1 00:00:00 _ subprocess1 998 tty1 00:00:00 _ mainprocess 1001 tty1 00:00:00 _ subprocess2 1002 tty1 00:00:00 _ subprocess2 1003 tty1 00:00:00 _ subprocess2</pre>

Question 7

Use `ps tree` command with option that show the number of threads to each process.

[illegible]

Question 8

Use `renice` command to change priority level of one of process “subprocess1”.

Command
<code>Sudo renice -5 999</code>
Output
<pre>zhiming@secr2043:~\$ sudo renice -5 999 [sudo] password for zhiming: 999 (process ID) old priority 0, new priority -5</pre>

Question 9

Terminate all running processes with the name “mainprocess”.

Command
<code>killall -15 mainprocess</code>
Output
<pre>zhiming@secre2043:~\$ killall -15 mainprocess Main process (ID: 997) received signal: 15. Terminating... Main process (ID: 996) received signal: 15. Terminating... Main process (ID: 998) received signal: 15. Terminating... [1]+ Done ./mainprocess</pre>

Question 10

Write a short C or Python code (choose only one language) demonstrating multiprocessing with `fork()` and `wait()`. Compile and/or run the code. Show the output.

Source Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        printf("Child process: My PID is %d\n", getpid());
        sleep(2);
        printf("Child process: Exiting now.\n");
    } else {
        printf("Parent process: My PID is %d, waiting for child %d\n", getpid(), pid);
        wait(NULL);
        printf("Parent process: Child finished. Exiting parent.\n");
    }

    return 0;
}
```

Output:

```
zhiming@secur2043:~$ ./example
Parent process: My PID is 1111, waiting for child 1112
Child process: My PID is 1112
Child process: Exiting now.
Parent process: Child finished. Exiting parent.
```